

No.8-demo 5.ADC-DMA采样

由 有限状态姬

ID:1006

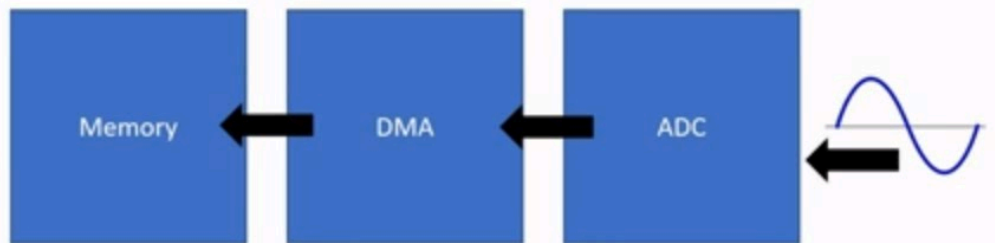
<https://www.emoe.xyz/bonjour-stm32-adc-dma/>

May 19, 2020

分类目录: Bonjour STM32, 嵌入式, 所有文章, 硬件

标签: Bonjour STM32, STM32, 教程

ADC DMA ADC with DMA



本文目录

给萌新们的demo 5.ADC-DMA
采样

- 1.前言
- 2.ADC是什么
- 3.STM32内置ADC介绍
- 4.建立ADC采样工程
- 5.总结

给萌新们的demo 5.ADC-DMA采样

上一章我们学习了UART通信和DMA传输（按理说是这样的，但是写上一篇DEMO的某只鸽子咕咕咕了，为了保证教程内容的完整性，我就假设大家在梦里学会了相关的内容），而本章内容将在上一章教程的基础上继续学习如何用单片机内置ADC采集输入电压值。

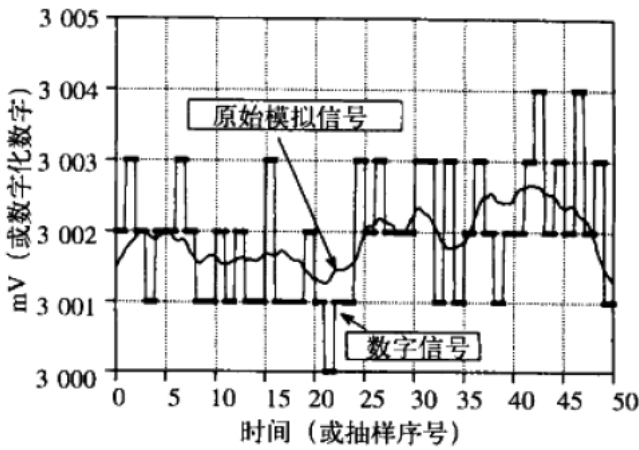
1.前言

本章依然是用淘宝某爆款STM32F103C8T6核心板作为开发平台，下面出现的原理图均以此为准。本章讲解的内容有以下几点：

什么是模数转换器（ADC）
了解STM32片内ADC的组成
*如何用STM32实现ADC转换并将结果输出

2.ADC是什么

ADC（普通攻击持续输出核心，模数转换器），即用来把模拟信号转成数字信号的电子器件。而由于单片机是由时钟信号驱动来工作的，使得它只能处理一个个离散的数据，因此需要有一个器件把外界的连续信号转换为单片机可以处理的离散信号。而且单片机不仅只能接受离散的时域信号，由于计算机系统本身的结构，单片机接收的数值也不是连续的，因此人们把这种在时间和数值上都是离散的信号叫做数字信号。



这样为了衡量ADC在数模转换过程中的性能，人们就用**采样率**和**分辨率**来分别描述ADC转换成的数字信号在时域和幅值这两个方面的性能。采样率表述的是ADC在单位时间内对连续信号的采样次数，分辨率则表示在输出允许的范围内把能输出的离散数值的个数。在工程中，我们用比特（bit）来表述ADC的分辨率（假设一个ADC的的分辨率为8bit，那么能输出的最小值电压就是 $V_{ref}/(2^8)=V_{ref}/256$ ），用SPS（每秒采样次数）来表述采样率。

当我们观察ADC输出的转换结果，却发现它输出的只是一个整数，和我们输入的信号数值相差甚远，那么我们该如何才能得到正确的采样结果呢？假设我们的ADC分辨率为a bit，ADC的最大采样电压为Vref，Din表示我们的转换输出结果，那么转换公式为：

$$V = D_{in} \times \frac{V_{ref}}{2^a}$$

有的时候我们需要通过ADC转换得到的数据重新通过DAC还原为原来的连续信号或者尽可能保留连续信号的信息，那么在采样时我们就要对ADC的采样率有所要求。一般来说**ADC的采样率要大于你要采样的连续信号的最高频率的两倍**，这个频率称为**奈奎斯特采样频率**，有关奈奎斯特采样频率和奈奎斯特采样定律的更多知识我们就不在这里过多讲述了，有兴趣的可以自行查阅有关“信号与系统”和“数字信号处理”的资料和课程。

3.STM32内置ADC介绍

从上面的内容我们了解到ADC的采样率和分辨率对ADC的性能影响非常大，它直接影响了我们是否能尽可能地准确保留采样信号的原有信息。如果现有的ADC不能很好地满足你的要求，那么你就需要思考一下用别的方法来满足，比如氪金来买更优秀的ADC，但它们高性能的背后所带来的就是对ADC的时序要求更高（简而言之就是更难用）。所幸的是对于初学的萌新来说，STM32的内置ADC完全可以满足我们前期的学习需求，虽然尚方宝剑削铁如泥，但是萌新用全村最好的剑砍砍村口的鸡，掌握住基本技能就足够了，说不定哪天就成了十里坡剑神（并不）。那么我们接下来就简要介绍一下单片机的内置ADC。

在[STM32F1数据手册](#)的第11章主要讲解了内置ADC的性能参数、功能描述、工作模式以及涉及到的内部寄存器操作等内容。在这里我们不讲比较复杂的内部功能图以及寄存器操作等内容，仅对我们比较关心的内容略作讲解。

11.2 ADC主要特征

- 12位分辨率
- 转换结束、注入转换结束和发生模拟看门狗事件时产生中断
- 单次和连续转换模式
- 从通道0到通道n的自动扫描模式
- 自校准
- 带内嵌数据一致性的数据对齐
- 采样间隔可以按通道分别编程
- 规则转换和注入转换均有外部触发选项
- 间断模式
- 双重模式(带2个或以上ADC的器件)
- ADC转换时间：
 - STM32F103xx增强型产品：时钟为56MHz时为1μs(时钟为72MHz为1.17μs)
 - STM32F101xx基本型产品：时钟为28MHz时为1μs(时钟为36MHz为1.55μs)
 - STM32F102xxUSB型产品：时钟为48MHz时为1.2μs
 - STM32F105xx和STM32F107xx产品：时钟为56MHz时为1μs(时钟为72MHz为1.17μs)
- ADC供电要求：2.4V到3.6V
- ADC输入范围： $V_{REF-} \leq V_{IN} \leq V_{REF+}$
- 规则通道转换期间有DMA请求产生。

11.6 可编程的通道采样时间

ADC使用若干个ADC_CLK周期对输入电压采样，采样周期数目可以通过ADC_SMPR1和ADC_SMPR2寄存器中的SMP[2:0]位更改。每个通道可以分别用不同的时间采样。

总转换时间如下计算：

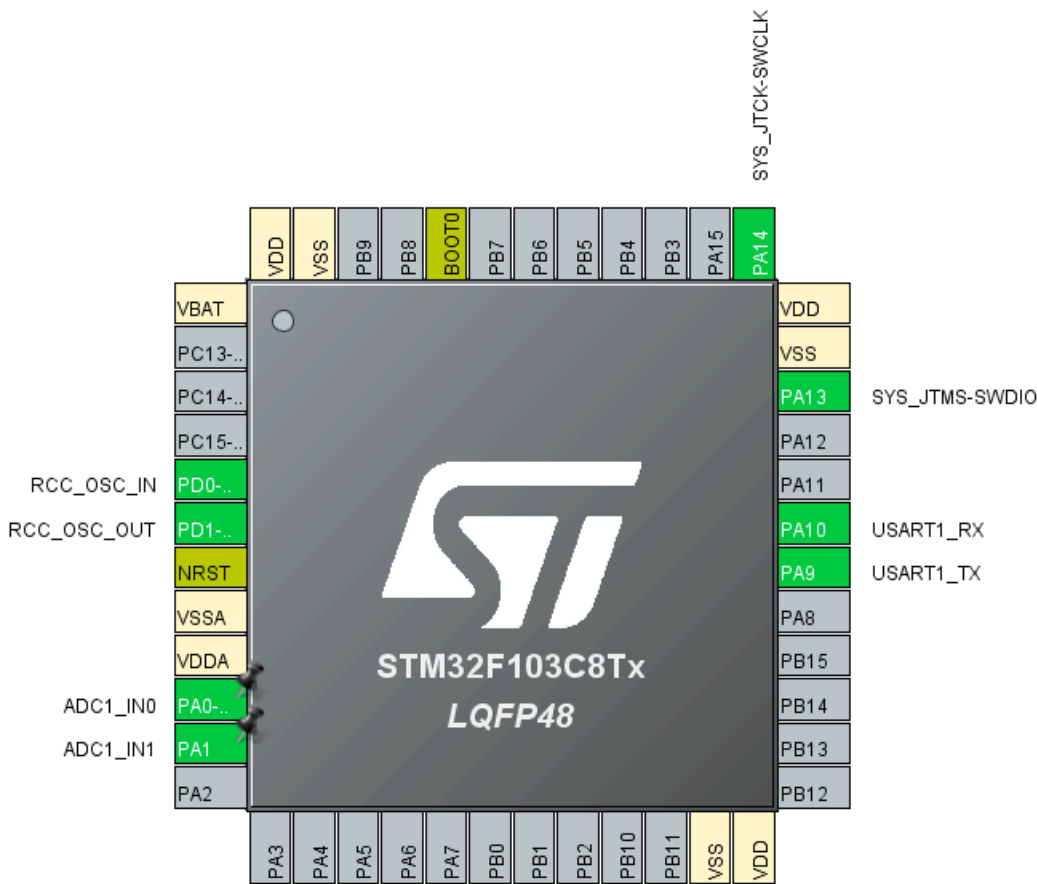
$$T_{CONV} = \text{采样时间} + 12.5\text{个周期}$$

上面是数据手册中列出的ADC的主要参数，我们可以看到这是一个12位ADC，因此在使用DMA传输时的长度应选择半字或一个字长。而采样率是可调的，本工程每个通道的采样时间选择1.5个周期，ADC时钟频率为12MHz，这样我们可以计算出本工程使用的ADC采样率约为860KHz。

如果有使用其他单片机的萌新可能会有疑惑，为什么有的单片机引脚上有Vref输入，而STM32F103C8T6却没有？如果是有Vref输入的单片机，那么ADC的输入信号范围就由那个参考电压输入引脚来决定；如果没有Vref引脚，那么单片机内部会把参考电压接到单片机输入电压上，那么这里的Vref=3.3V。

4.建立ADC采样工程

打开CubeMX新建工程，这里我们把PA0和PA1设置为ADC1的两个通道IN0和IN1，PA9和PA10设置为USART1的发送端和接收端，用于发送ADC采集到的数据。



然后对ADC进行配置。设置ADC通道数为2；数据右对齐；使能扫描转换模式和连续转换模式；将每个通道的转换时间设置为1.5个周期。

▼ ADCs_Common_Settings	Mode	Independent mode
▼ ADC_Settings		
	Data Alignment	Right alignment
	Scan Conversion Mode	Enabled
	Continuous Conversion Mode	Enabled
	Discontinuous Conversion Mode	Disabled
▼ ADC_Regular_ConversionMode		
	Enable Regular Conversions	Enable
	Number Of Conversion	2
	External Trigger Conversion Source	Regular Conversion launched by software
▼	Rank	1
	Channel	Channel 0
	Sampling Time	1.5 Cycles
▼	Rank	2
	Channel	Channel 1
	Sampling Time	1.5 Cycles
▼ ADC_Injected_ConversionMode		
	Enable Injected Conversions	Disable
▼ WatchDog		
	Enable Analog WatchDog Mode	☐

打开DMA配置界面，添加DMA设置，设置为连续传输模式，传输数据字长设置为1个字长。

DMA Request	Channel	Direction	Priority
ADC1	DMA1 Channel 1	Peripheral To Memory	Low

Add
Delete

DMA Request Settings

Mode	Circular	Increment Address	☐	Peripheral	☐	Memory	☒
		Data Width	Word			Word	

打开USART1配置界面，设置波特率为115200bit/s，8位数据长度，1位停止位，无校验位。

Mode

Hardware Flow Control (RS232)

Configuration

☒ Parameter Settings
 ☒ User Constants
 ☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings

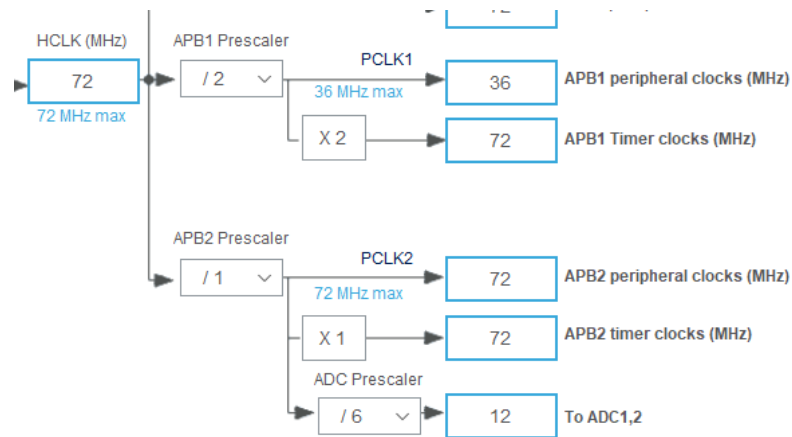
Configure the below parameters :

▼ Basic Parameters

Baud Rate	115200 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1

▼ Advanced Parameters

最后我们打开时钟配置界面，可以看到ADC时钟频率是将单片机工作频率经过6分频得到的。



随后建立工程，用MDK-ARM打开工程界面。首先在主函数前面添加一个数据缓存数组ADCbuffer，用于缓存从DMA传输过来的数据；然后新建两个字符串数组，用于发送采样结果。

```

/* USER CODE BEGIN PV */
/*字符串数组用于保存转换后的字符串显示结果*/
char ADC_1c;
char ADC_2c;
/*数据缓存数组用于保存两个通道的电压值*/
volatile uint32_t ADCbuffer;
/* USER CODE END PV */
  
```

之后在while(1)之前添加两个函数，分别用来对ADC进行校准和开启ADC-DMA模式转换。

```
/* USER CODE BEGIN 2 */
/*开启转换前先进行ADC校准，然后以ADC_DMA模式打开转换*/
HAL_ADCEx_Calibration_Start(&hadc1);
HAL_ADC_Start_DMA(&hadc1,(uint32_t *)ADCbuffer,2);
/* USER CODE END 2 */
```

由于开启了扫描转换模式，因此缓冲区ADCbuffer和ADCbuffer分别保存了ADC1_0和ADC_1通道的数据。而且串口通信只能传输字符类型数据，我们还要使用sprintf()函数来将采集数据转换成字符串。在使用sprintf()之前记得声明stdio.h头文件。

```
while (1)
{
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
/*换算公式：V=Din*Vref/(2^12)*/
/*将ADC转换结果转换成字符串形式*/
sprintf(ADC_1c,"ADC_1=%.5fV",ADCbuffer*3.3f/4096);
sprintf(ADC_2c,"ADC_2=%.5fV",ADCbuffer*3.3f/4096);

HAL_UART_Transmit(&huart1,(uint8_t*)ADC_1c,15,1000);
HAL_UART_Transmit(&huart1,(uint8_t*)('\t'),1,1000);
HAL_UART_Transmit(&huart1,(uint8_t*)ADC_2c,15,1000);
HAL_UART_Transmit(&huart1,(uint8_t*)('\n'),1,1000);

HAL_Delay(500);
}
/* USER CODE END 3 */
```

这样一个简单的ADC采集并转换的工程就完成了。

5.总结

本文我们讲解了如何用STM32片内ADC实现对电压的采集与转换，现将步骤总结如下：

- 1.将相应的端口配置为ADC采样通道，配置ADC的采样模式。
- 2.添加DMA，并把DMA传输设置成连续传输模式。
- 3.使用函数HAL_ADCEx_Calibration_Start()对ADC进行校准。
- 4.使用HAL_ADC_Start_DMA()开启转换。